

Nonlinear Conjugate Gradient With Perturbed Heavy-Ball Saddle Escape

Benjamin Séguin

Université Paris Dauphine-PSL

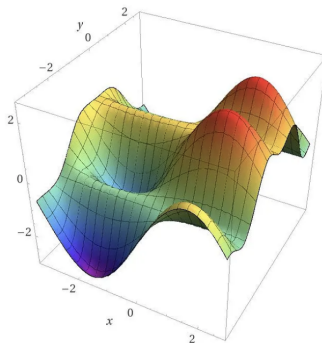
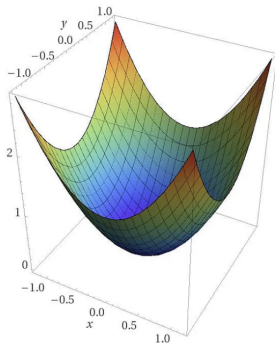
Supervised by Clément W. Royer

Master's thesis defence, 10th September 2026

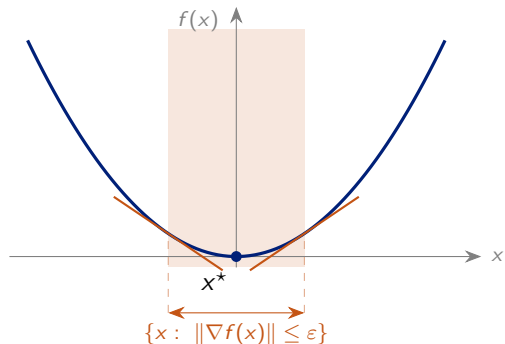
$$\min_{x \in \mathbb{R}^n} f(x)$$

$$\min_{x \in \mathbb{R}^n} f(x)$$

Algorithms have different guarantees depending on whether f is convex or nonconvex.



Convex optimization: a small gradient is enough



the tangents at the edges of the band have slope $\pm\varepsilon$

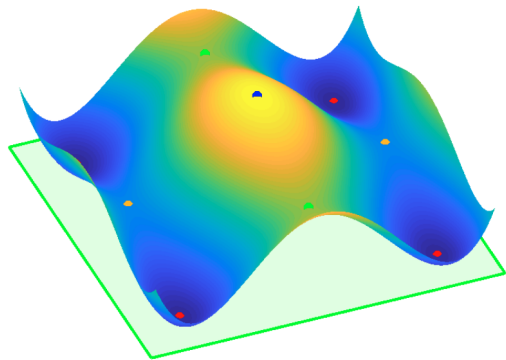
Convex functions have only one global minimum at which $\nabla f(x) = 0$.

We look for an ε -first-order stationary point (FOSP):

$$\|\nabla f(x)\| \leq \varepsilon.$$

As $\varepsilon \rightarrow 0$ we certify a local minimum, and every local minimum is **global**.

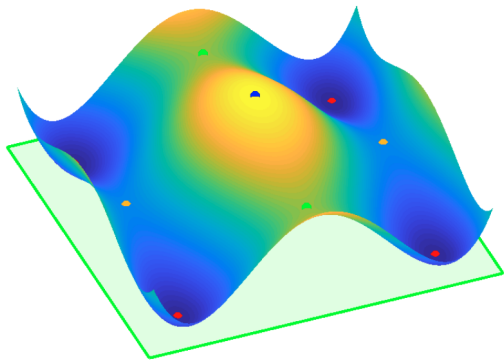
Nonconvex: a small gradient is not enough



Wright & Ma '22

There are **many** points with $\nabla f(x) = 0$, and not all of them are minima. The others are local maxima and saddle points.

Nonconvex: a small gradient is not enough



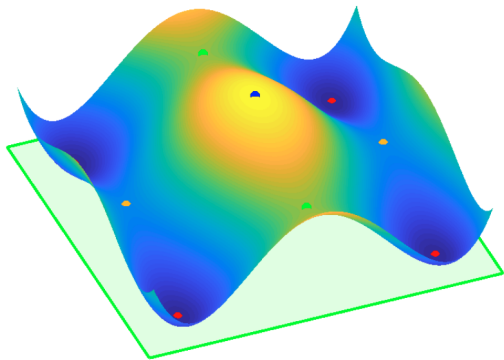
Wright & Ma '22

There are **many** points with $\nabla f(x) = 0$, and not all of them are minima. The others are local maxima and saddle points.

We therefore ask for an $(\varepsilon_g, \varepsilon_H)$ -second-order stationary point (SOSP):

$$\|\nabla f(x)\| \leq \varepsilon_g \quad \text{and} \quad \lambda_{\min}(\nabla^2 f(x)) \geq -\varepsilon_H.$$

Nonconvex: a small gradient is not enough



Wright & Ma '22

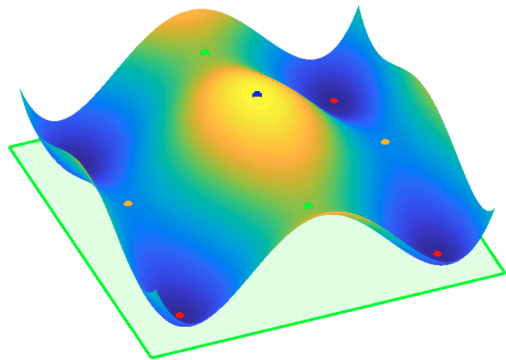
There are **many** points with $\nabla f(x) = 0$, and not all of them are minima. The others are local maxima and saddle points.

We therefore ask for an $(\varepsilon_g, \varepsilon_H)$ -second-order stationary point (SOSP):

$$\|\nabla f(x)\| \leq \varepsilon_g \quad \text{and} \quad \lambda_{\min}(\nabla^2 f(x)) \geq -\varepsilon_H.$$

This rules out **nondegenerate** local maxima and saddle points, and is the standard target in nonconvex optimization. We use the special case $\varepsilon_H = \sqrt{\varepsilon_g}$ throughout.

Nonconvex: a small gradient is not enough



Wright & Ma '22

There are **many** points with $\nabla f(x) = 0$, and not all of them are minima. The others are local maxima and saddle points.

We therefore ask for an $(\varepsilon_g, \varepsilon_H)$ -second-order stationary point (SOSP):

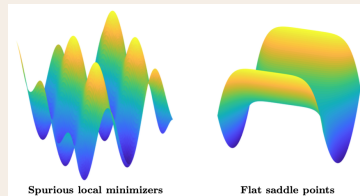
$$\|\nabla f(x)\| \leq \varepsilon_g \quad \text{and} \quad \lambda_{\min}(\nabla^2 f(x)) \geq -\varepsilon_H.$$

This rules out **nondegenerate** local maxima and saddle points, and is the standard target in nonconvex optimization. We use the special case $\varepsilon_H = \sqrt{\varepsilon_g}$ throughout.

Certifying an actual local, let alone global, minimum is **NP-hard** without further assumptions on the landscape. (Murty & Kabadi '87)

Bad instances

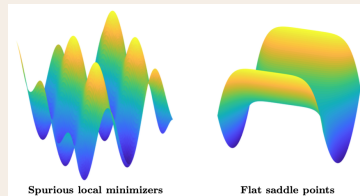
- Local, non-global minima.
- High-order saddle points.



Nonconvex optimization: a matter of landscape

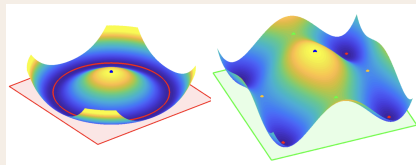
Bad instances

- Local, non-global minima.
- High-order saddle points.



Nice instances

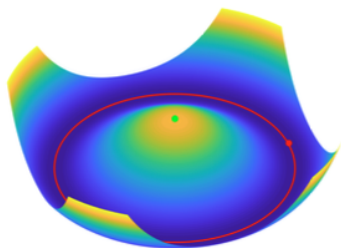
- All local minima are global.
- Strict (non-flat) saddle points.



Figures: J. Wright and Y. Ma, *High-Dimensional Data Analysis with Low-Dimensional Models*, 2022.

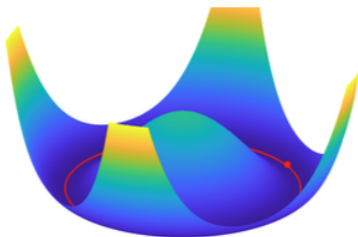
Generalized phase retrieval

$$\min_x \frac{1}{2} \|y^2 - |Ax|^2\|_2^2$$



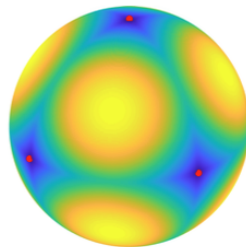
Matrix recovery

$$\min_{U,V} \mathcal{L}(Y - \mathcal{A}[UV^*]) + \rho(U, V)$$



Dictionary learning

$$\min_{A \in \mathcal{A}, X} \frac{1}{2} \|Y - AX\|_F^2 + \lambda \|X\|_1$$



Pictures: J. Wright and Y. Ma, High-Dimensional Data Analysis with Low-Dimensional Models, 2022.

Escaping saddle points cheaply

- Do it with **gradients only**, no Hessian.
- Do it for **nonlinear conjugate gradient (NCG)**, a practical method with limited theory in the nonconvex setting.
- Prove a **complexity guarantee**: how many iterations, in the worst case.

Escaping saddle points cheaply

- Do it with **gradients only**, no Hessian.
- Do it for **nonlinear conjugate gradient (NCG)**, a practical method with limited theory in the nonconvex setting.
- Prove a **complexity guarantee**: how many iterations, in the worst case.

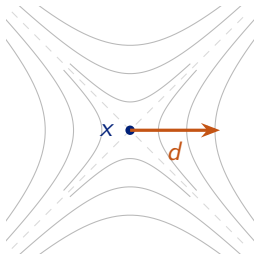
Main result

An algorithm that returns an $(\varepsilon_g, \varepsilon_H)$ -SOSP with high probability. To our knowledge, the first such guarantee for a method based on NCG.

1. How do we escape a saddle point?
2. Conjugate gradient and heavy-ball
3. The method and the result
4. Does it work?
5. Conclusion

1. How do we escape a saddle point?
2. Conjugate gradient and heavy-ball
3. The method and the result
4. Does it work?
5. Conclusion

Option 1: look at the curvature



level sets near a strict saddle

Negative curvature methods read the Hessian and step **along the direction of most negative curvature**.

- Best known rates: $O(\max\{\varepsilon_g^{-3/2}, \varepsilon_H^{-3}\})$, and this is sharp.

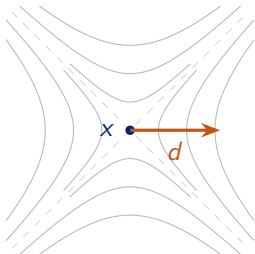
The Hessian $\nabla^2 f(x)$ collects the second derivatives and gives the local curvature:

$$f(x + d) \approx f(x) + \nabla f(x)^\top d + \frac{1}{2} d^\top \nabla^2 f(x) d.$$

d is a direction of **negative curvature** at x if

$$d^\top \nabla^2 f(x) d \leq -\varepsilon_H \|d\|^2.$$

Option 1: look at the curvature



level sets near a strict saddle

The Hessian $\nabla^2 f(x)$ collects the second derivatives and gives the local curvature:

$$f(x + d) \approx f(x) + \nabla f(x)^\top d + \frac{1}{2} d^\top \nabla^2 f(x) d.$$

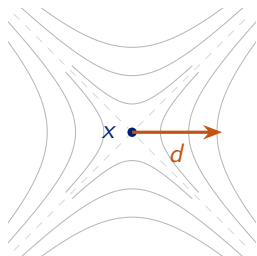
d is a direction of **negative curvature** at x if

$$d^\top \nabla^2 f(x) d \leq -\varepsilon_H \|d\|^2.$$

Negative curvature methods read the Hessian and step **along the direction of most negative curvature**.

- Best known rates: $O(\max\{\varepsilon_g^{-3/2}, \varepsilon_H^{-3}\})$, and this is sharp.
- But curvature is queried **at every iteration**, and each query costs several Hessian-vector products even in its cheapest form.

Option 1: look at the curvature



level sets near a strict saddle

The Hessian $\nabla^2 f(x)$ collects the second derivatives and gives the local curvature:

$$f(x + d) \approx f(x) + \nabla f(x)^\top d + \frac{1}{2} d^\top \nabla^2 f(x) d.$$

d is a direction of **negative curvature** at x if

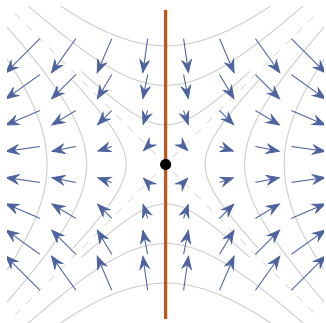
$$d^\top \nabla^2 f(x) d \leq -\varepsilon_H \|d\|^2.$$

Negative curvature methods read the Hessian and step **along the direction of most negative curvature**.

- Best known rates: $O(\max\{\varepsilon_g^{-3/2}, \varepsilon_H^{-3}\})$, and this is sharp.
- But curvature is queried **at every iteration**, and each query costs several Hessian-vector products even in its cheapest form.

In high dimension, that is what one would like to avoid. Nesterov & Polyak '06; Cartis et al. '11; Royer et al. '20; Curtis et al. '21.

Option 2: hope that gradient descent escapes fast

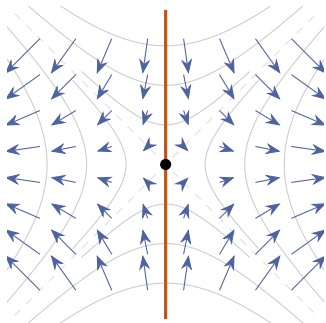


the initializations that get stuck

The gradient flow near a saddle pushes almost every point away. Only the initialisations on a lower-dimensional set converge to it.

- GD avoids strict saddles from almost every starting point.

Option 2: hope that gradient descent escapes fast

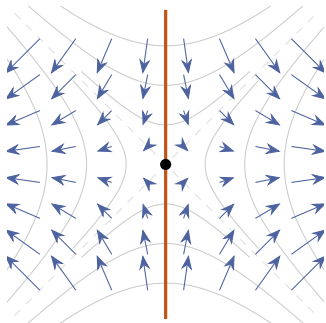


the initializations that get stuck

The gradient flow near a saddle pushes almost every point away. Only the initialisations on a lower-dimensional set converge to it.

- GD avoids strict saddles from almost every starting point.
- But the guarantee is asymptotic: no bound on how long it takes.

Option 2: hope that gradient descent escapes fast



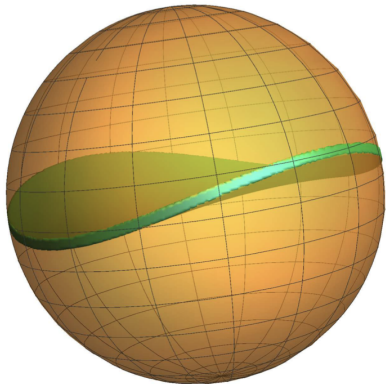
the initializations that get stuck

The gradient flow near a saddle pushes almost every point away. Only the initialisations on a lower-dimensional set converge to it.

- GD avoids strict saddles from almost every starting point.
- But the guarantee is asymptotic: no bound on how long it takes.
- And there are functions on which it takes exponentially many iterations to get away.

Lee et al. '16, '19; Musat & Boumal '25; Du et al. '17.

Option 3: add a little noise

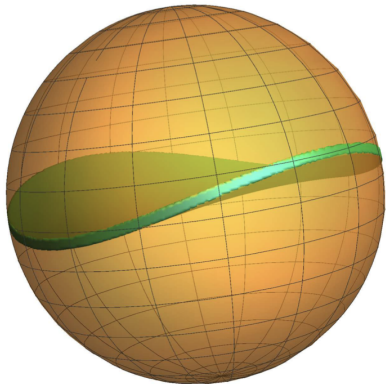


the ball of perturbations

When the gradient becomes small, **perturb the iterate** inside a small ball and keep going for a fixed number of steps.

- The perturbations that fail to escape form a **thin pancake**.

Option 3: add a little noise

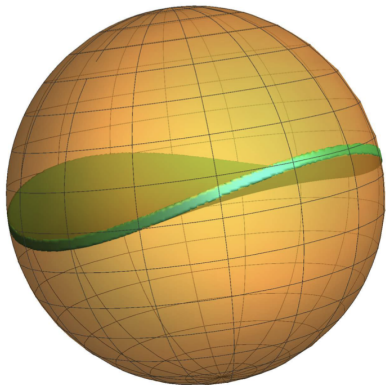


the ball of perturbations

When the gradient becomes small, **perturb the iterate** inside a small ball and keep going for a fixed number of steps.

- The perturbations that fail to escape form a **thin pancake**.
- Perturbed gradient descent (PGD) and perturbed accelerated gradient descent (PAGD) come from this framework.

Option 3: add a little noise



the ball of perturbations

When the gradient becomes small, **perturb the iterate** inside a small ball and keep going for a fixed number of steps.

- The perturbations that fail to escape form a **thin pancake**.
- Perturbed gradient descent (PGD) and perturbed accelerated gradient descent (PAGD) come from this framework.
- Result: an $(\varepsilon_g, \varepsilon_H)$ -SOSP in $\tilde{O}(\varepsilon_g^{-2})$ iterations for PGD, $\tilde{O}(\varepsilon_g^{-7/4})$ for PAGD.

Picture after Jin, Ge, Netrapalli, Kakade & Jordan '17; Jin, Netrapalli & Jordan '18.

Recall that a SOSP is defined as a point x such that:

$$\|\nabla f(x)\| \leq \varepsilon_g \quad \text{and} \quad \lambda_{\min}(\nabla^2 f(x)) \geq -\varepsilon_H.$$

In the table below, $\varepsilon_H = \sqrt{\varepsilon_g}$

	needs curvature?	reaches an $(\varepsilon_g, \varepsilon_H)$ -SOSP?	rate
Neg curvature	yes	yes	$\varepsilon_g^{-3/2}$
GD	no	asymptotically, no rate	—
PGD	no	yes, w.h.p.	$\tilde{O}(\varepsilon_g^{-2})$
PAGD	no	yes, w.h.p.	$\tilde{O}(\varepsilon_g^{-7/4})$

Recall that a SOSP is defined as a point x such that:

$$\|\nabla f(x)\| \leq \varepsilon_g \quad \text{and} \quad \lambda_{\min}(\nabla^2 f(x)) \geq -\varepsilon_H.$$

In the table below, $\varepsilon_H = \sqrt{\varepsilon_g}$

	needs curvature?	reaches an $(\varepsilon_g, \varepsilon_H)$ -SOSP?	rate
Neg curvature	yes	yes	$\varepsilon_g^{-3/2}$
GD	no	asymptotically, no rate	—
PGD	no	yes, w.h.p.	$\tilde{O}(\varepsilon_g^{-2})$
PAGD	no	yes, w.h.p.	$\tilde{O}(\varepsilon_g^{-7/4})$
NCG	no	unknown	—

1. How do we escape a saddle point?
2. Conjugate gradient and heavy-ball
3. The method and the result
4. Does it work?
5. Conclusion

Throughout, $g_k := \nabla f(x_k)$.

Heavy-ball

$$x_{k+1} = x_k - \alpha g_k + \gamma (x_k - x_{k-1}).$$

- Most classical momentum technique.
- $\gamma = 0 \rightarrow$ recover GD.
- Reuses the previous displacement.
- α and γ are **constants**, fixed in advance.

Conjugate gradient and heavy-ball

Throughout, $g_k := \nabla f(x_k)$.

Heavy-ball

$$x_{k+1} = x_k - \alpha g_k + \gamma (x_k - x_{k-1}).$$

- Most classical momentum technique.
- $\gamma = 0 \rightarrow$ recover GD.
- Reuses the previous displacement.
- α and γ are **constants**, fixed in advance.

Nonlinear conjugate gradient

$$x_{k+1} = x_k + \alpha_k d_k, \quad d_{k+1} = -g_{k+1} + \beta_{k+1} d_k.$$

- α_k from an Armijo line search.
- β_{k+1} a **nonlinear** function of two gradients:

$$\beta_{k+1}^{\text{PRP}^+} = \max \left\{ \frac{g_{k+1}^\top (g_{k+1} - g_k)}{\|g_k\|^2}, 0 \right\}.$$

Conjugate gradient and heavy-ball

Throughout, $g_k := \nabla f(x_k)$.

Heavy-ball

$$x_{k+1} = x_k - \alpha g_k + \gamma (x_k - x_{k-1}).$$

- Most classical momentum technique.
- $\gamma = 0 \rightarrow$ recover GD.
- Reuses the previous displacement.
- α and γ are **constants**, fixed in advance.

Nonlinear conjugate gradient

$$x_{k+1} = x_k + \alpha_k d_k, \quad d_{k+1} = -g_{k+1} + \beta_{k+1} d_k.$$

- α_k from an Armijo line search.
- β_{k+1} a **nonlinear** function of two gradients:

$$\beta_{k+1}^{\text{PRP}^+} = \max \left\{ \frac{g_{k+1}^\top (g_{k+1} - g_k)}{\|g_k\|^2}, 0 \right\}.$$

Both reuse past information at the cost of **one gradient per iteration**.

Polyak '64; Fletcher & Reeves '64; Polak–Ribière–Polyak '69.

Nonlinear conjugate gradient has thin theoretical guarantees

d_{k+1} may not be a descent direction, so implementations **restart**: throw the direction away and set $d_{k+1} = -g_{k+1}$.

Nonlinear conjugate gradient has thin theoretical guarantees

d_{k+1} may not be a descent direction, so implementations **restart**: throw the direction away and set $d_{k+1} = -g_{k+1}$.

A restart test with guarantees

Given $\sigma \in (0, 1]$, $\kappa \geq 1$, $p, q \geq 0$, keep d_{k+1} only if

$$\underbrace{g_{k+1}^\top d_{k+1} \leq -\sigma \|g_{k+1}\|^{1+p}}_{\text{the direction descends}} \quad \text{and} \quad \underbrace{\|d_{k+1}\| \leq \kappa \|g_{k+1}\|^q}_{\text{its norm stays controlled}}$$

This gives the first worst-case complexity for NCG in the nonconvex setting, but only to an ε_g -FOSP.

No NCG method is known to reach an $(\varepsilon_g, \varepsilon_H)$ -SOSP.

1. How do we escape a saddle point?
2. Conjugate gradient and heavy-ball
3. The method and the result
4. Does it work?
5. Conclusion

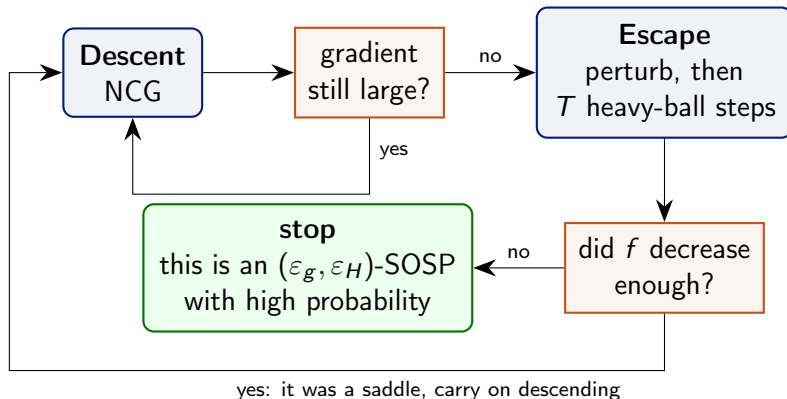
Our proposal: a two phase method

A **descent phase** reaches a FOSP, which might be a saddle point, and an **escape phase** tries to escape it.

	what it requires	what provides it
Descent	fast progress while the gradient is large ($\ g_k\ \geq \varepsilon_g$)	NCG
Escape	fixed parameters inside any given run (needed for proof technique)	heavy-ball

Algorithm diagram

Our algorithm **NCG-HB** is:



The decrease in f over one escape window is the **certificate**: no decrease means no negative curvature, so no escape was possible.

Theorem

Under Lipschitz gradient and Lipschitz Hessian, the method stops and returns a point that is an $(\varepsilon_g, \varepsilon_H)$ -SOSP with probability at least $1 - \delta$, after at most

$$\tilde{O}\left(\underbrace{\varepsilon_g^{-2} + \varepsilon_g^{-\max\{1+p, 2(1+p-q)\}}}_{\text{descent phase}} + \underbrace{\varepsilon_H^{-4}}_{\text{escape phase}}\right) \quad \text{iterations.}$$

Theorem

Under Lipschitz gradient and Lipschitz Hessian, the method stops and returns a point that is an $(\varepsilon_g, \varepsilon_H)$ -SOSP with probability at least $1 - \delta$, after at most

$$\tilde{O}\left(\underbrace{\varepsilon_g^{-2} + \varepsilon_g^{-\max\{1+p, 2(1+p-q)\}}}_{\text{descent phase}} + \underbrace{\varepsilon_H^{-4}}_{\text{escape phase}}\right) \text{ iterations.}$$

- p, q are the parameters of the restart test, typically in $[0, 1]$. The ε_g^{-2} is only for **restart steps**; accepted NCG steps correspond to the second rate, which is $\varepsilon_g^{-3/2}$ for $p = \frac{1}{2}, q = 1$.

Theorem

Under Lipschitz gradient and Lipschitz Hessian, the method stops and returns a point that is an $(\varepsilon_g, \varepsilon_H)$ -SOSP with probability at least $1 - \delta$, after at most

$$\tilde{O}\left(\underbrace{\varepsilon_g^{-2} + \varepsilon_g^{-\max\{1+p, 2(1+p-q)\}}}_{\text{descent phase}} + \underbrace{\varepsilon_H^{-4}}_{\text{escape phase}}\right) \text{ iterations.}$$

- p, q are the parameters of the restart test, typically in $[0, 1]$. The ε_g^{-2} is only for **restart steps**; accepted NCG steps correspond to the second rate, which is $\varepsilon_g^{-3/2}$ for $p = \frac{1}{2}$, $q = 1$.
- With the usual coupling $\varepsilon_H = \sqrt{\varepsilon_g}$, the escape term is ε_g^{-2} : **the rate of PGD**.

Theorem

Under Lipschitz gradient and Lipschitz Hessian, the method stops and returns a point that is an $(\varepsilon_g, \varepsilon_H)$ -SOSP with probability at least $1 - \delta$, after at most

$$\tilde{O}\left(\underbrace{\varepsilon_g^{-2} + \varepsilon_g^{-\max\{1+p, 2(1+p-q)\}}}_{\text{descent phase}} + \underbrace{\varepsilon_H^{-4}}_{\text{escape phase}}\right) \quad \text{iterations.}$$

- p, q are the parameters of the restart test, typically in $[0, 1]$. The ε_g^{-2} is only for **restart steps**; accepted NCG steps correspond to the second rate, which is $\varepsilon_g^{-3/2}$ for $p = \frac{1}{2}$, $q = 1$.
- With the usual coupling $\varepsilon_H = \sqrt{\varepsilon_g}$, the escape term is ε_g^{-2} : **the rate of PGD**.
- First second-order guarantee for a method based on NCG.

1. How do we escape a saddle point?
2. Conjugate gradient and heavy-ball
3. The method and the result
4. Does it work?
5. Conclusion

Two test problems

A quadratic saddle

$$f(x) = \frac{1}{2}x^\top Hx + \frac{\nu}{4}\|x\|^4$$

Spectrum chosen by hand, so the curvature at the saddle is known exactly.

Matrix factorization

$$f(U) = \frac{1}{4}\|UU^\top - M\|_F^2$$

Every local minimum global, every saddle strict. The origin is a saddle.

- All methods share the same stepsize and the same escape budget. Nothing is tuned.
- Escape runs **start on a saddle**: from a random point one is essentially never met, and there would be nothing to measure.
- Two variants of NCG-HB are compared against GD, PGD, PAGD.
- All numbers are medians over 20 repetitions.

The descent phase is fast

ε_g	quadratic saddle				matrix factorization			
	10^{-2}	10^{-4}	10^{-6}	10^{-8}	10^{-2}	10^{-4}	10^{-6}	10^{-8}
GD	109	144	174	204	1594	1989	2469	3234
NCG	4	15	20	26	20	26	35	47

- Iterations to reach a small gradient. **47** against 3234.
- The method never restarts here: these are real NCG steps.

method	quadratic saddle		matrix factorization	
	iterations	evaluations	iterations	evaluations
GD	never	—	6248	6249
perturbed NCG	175	181	970	1015
PGD	210	213	2181	2187
PAGD	149	596	570	2280
NCG-HB	99	105	956	967
NCG-HB, with early exit	50	68	137	203

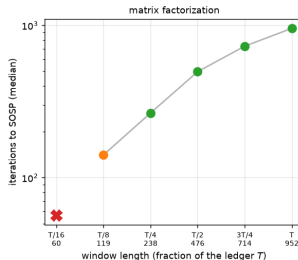
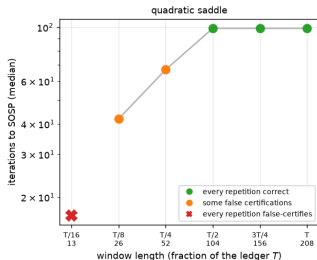
method	quadratic saddle		matrix factorization	
	iterations	evaluations	iterations	evaluations
GD	never	—	6248	6249
perturbed NCG	175	181	970	1015
PGD	210	213	2181	2187
PAGD	149	596	570	2280
NCG-HB	99	105	956	967
NCG-HB, with early exit	50	68	137	203

- In iterations: NCG-HB ahead of PGD on both, ahead of the accelerated version on one.

method	quadratic saddle		matrix factorization	
	iterations	evaluations	iterations	evaluations
GD	never	—	6248	6249
perturbed NCG	175	181	970	1015
PGD	210	213	2181	2187
PAGD	149	596	570	2280
NCG-HB	99	105	956	967
NCG-HB, with early exit	50	68	137	203

- In iterations: NCG-HB ahead of PGD on both, ahead of the accelerated version on one.
- In **evaluations of f and ∇f** ahead of both, on both. One accelerated iteration costs four evaluations; one window step costs one.

The early exit version

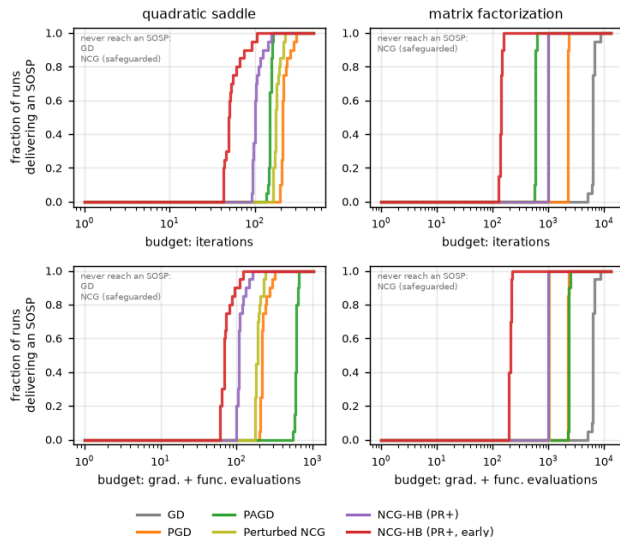


The escape window is sized for the worst case. In practice the escape is over after a fraction of its window's size, the rest is spent waiting for the budget to run out.

An **early exit** leaves the escape phase as soon as the sufficient drop in f has been observed.

Reaching a SOSP for matrix factorization: $956 \rightarrow 137$ iterations.

Convergence comparison



Our method with early exit beats the others in all configuration.

PAGD degrades badly when the budget is in evaluations.

1. How do we escape a saddle point?
2. Conjugate gradient and heavy-ball
3. The method and the result
4. Does it work?
5. Conclusion

- We derive a **second-order complexity guarantee** for an NCG-based method for the first time.
- The worst-case bound matches that of PGD, but can improve in certain regimes.
- Empirically the method reaches good points in fewer evaluations than both PGD and PAGD, on both test problems.
- In the last chapter of the thesis, we report complexity guarantees for convergence to a **local minimum** under an additional assumption on the landscape.

Short term

- Does NCG itself escape saddle points? Not answered here: it is used only to descend.
- The two phases are independent. Any descent method, any escape mechanism: which pairs work best, and does it depend on the problem?

Short term

- Does NCG itself escape saddle points? Not answered here: it is used only to descend.
- The two phases are independent. Any descent method, any escape mechanism: which pairs work best, and does it depend on the problem?

Longer term

This becomes the first chapter of a PhD, on exploiting landscape structure in nonconvex optimization, with applications to scientific imaging and inverse problems.

Thank you.

Questions

Closest related work

- R. Chan–Renous–Legoubin and C. W. Royer. *A nonlinear conjugate gradient method with complexity guarantees and its application to nonconvex regression*. EJCO, 2022.
- C. Jin, R. Ge, P. Netrapalli, S. M. Kakade and M. I. Jordan. *How to escape saddle points efficiently*. ICML, 2017.
- C. Jin, P. Netrapalli and M. I. Jordan. *Accelerated gradient descent escapes saddle points faster than gradient descent*. COLT, 2018.
- H. Li and Z. Lin. *Restarted nonconvex accelerated gradient descent: no more polylogarithmic factor in the $O(\epsilon^{-7/4})$ complexity*. JMLR, 2023.
- M. O’Neill and S. J. Wright. *Behavior of accelerated gradient methods near critical points of nonconvex functions*. Math. Program., 2019.

Saddle points and landscapes

- K. G. Murty and S. N. Kabadi. *Some NP-complete problems in quadratic and nonlinear programming*. Math. Program., 1987.

- Y. N. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli and Y. Bengio. *Identifying and attacking the saddle point problem in high-dimensional nonconvex optimization*. NeurIPS, 2014.
- R. Ge, F. Huang, C. Jin and Y. Yuan. *Escaping from saddle points — online stochastic gradient for tensor decomposition*. COLT, 2015.
- J. D. Lee, M. Simchowitz, M. I. Jordan and B. Recht. *Gradient descent only converges to minimizers*. COLT, 2016.
- J. D. Lee, I. Panageas, G. Piliouras, M. Simchowitz, M. I. Jordan and B. Recht. *First-order methods almost always avoid strict saddle points*. Math. Program., 2019.
- S. Du, C. Jin, J. D. Lee, M. I. Jordan, A. Singh and B. Póczos. *Gradient descent can take exponential time to escape saddle points*. NeurIPS, 2017.
- A.-A. Muşat and N. Boumal. *Gradient descent avoids strict saddles with a simple line-search method too*. Preprint, 2025.

Second-order and hybrid methods

- Y. Nesterov and B. T. Polyak. *Cubic regularization of Newton method and its global performance*. Math. Program., 2006.
- C. Cartis, N. I. M. Gould and P. L. Toint. *Adaptive cubic regularisation methods for unconstrained optimization. Part I*. Math. Program., 2011.
- C. W. Royer, M. O'Neill and S. J. Wright. *A Newton-CG algorithm with complexity guarantees for smooth unconstrained optimization*. Math. Program., 2020.
- F. E. Curtis, D. P. Robinson, C. W. Royer and S. J. Wright. *Trust-region Newton-CG with strong second-order complexity guarantees for nonconvex optimization*. SIAM J. Optim., 2021.

Conjugate gradient and momentum

- B. T. Polyak. *Some methods of speeding up the convergence of iteration methods*. USSR Comput. Math. and Math. Phys., 1964.
- R. Fletcher and C. M. Reeves. *Function minimization by conjugate gradients*. The Computer Journal, 1964.
- E. Polak and G. Ribière. *Note sur la convergence de méthodes de directions conjuguées*. R.I.R.O., 1969.

Figures

- J. Wright and Y. Ma. *High-Dimensional Data Analysis with Low-Dimensional Models*. Cambridge University Press, 2022.

```

1: Input:  $x_0; L_1, L_2; \varepsilon_g, \varepsilon_H$ ; Armijo  $\eta, \theta$ ; restart parameters  $\sigma, \kappa, p, q$ ; momentum  $\gamma < \frac{1}{2}$ ;  $\delta_{\text{tot}}; \Delta_f$ ;  $\alpha_w \leftarrow 1/L_1$ 
2: Derived: window length  $T$ , radius  $R$ , target decrease  $F$ , slack  $\omega$ ;  $F_{\text{cert}} \leftarrow (1 - \omega)F^2$ 
3:  $g_0 \leftarrow \nabla f(x_0)$ ;  $d_0 \leftarrow -g_0$ 
4: for  $k = 0, 1, 2, \dots$  do
5:   if  $\|g_k\| \leq \varepsilon_g$  then                                     ▷ candidate SOSP: perturb and run a window
6:      $\tilde{x} \leftarrow x_k$ ;  $\hat{x}_0 \leftarrow \tilde{x} + \xi$ ,  $\xi \sim \text{Unif}(B_0(R))$ ;  $\hat{x}_{-1} \leftarrow \hat{x}_0$ 
7:     for  $t = 0, \dots, T - 1$  do
8:        $\hat{x}_{t+1} \leftarrow \hat{x}_t - \alpha_w \nabla f(\hat{x}_t) + \gamma(\hat{x}_t - \hat{x}_{t-1})$ 
9:     end for
10:    if  $f(\tilde{x}) - f(\hat{x}_T) \geq F_{\text{cert}}$  then                               ▷ escaped:  $\tilde{x}$  was a strict saddle
11:       $x_{k+1} \leftarrow \hat{x}_T$ ;  $g_{k+1} \leftarrow \nabla f(\hat{x}_T)$ ;  $d_{k+1} \leftarrow -g_{k+1}$ 
12:    else
13:      return  $\tilde{x}$                                                          ▷ an  $(\varepsilon_g, \varepsilon_H)$ -SOSP with high probability
14:    end if
15:  else                                                         ▷ descent: safeguarded NCG
16:     $\alpha_k \leftarrow \theta^{j_k}$ ,  $j_k$  minimal with  $f(x_k + \theta^{j_k} d_k) \leq f(x_k) + \eta \theta^{j_k} g_k^\top d_k$ 
17:     $x_{k+1} \leftarrow x_k + \alpha_k d_k$ ;  $g_{k+1} \leftarrow \nabla f(x_{k+1})$ ;  $d_{k+1}^{\text{trial}} \leftarrow -g_{k+1} + \beta_{k+1} d_k$ 
18:     $d_{k+1} \leftarrow d_{k+1}^{\text{trial}}$  if  $g_{k+1}^\top d_{k+1}^{\text{trial}} \leq -\sigma \|g_{k+1}\|^{1+p}$  and  $\|d_{k+1}^{\text{trial}}\| \leq \kappa \|g_{k+1}\|^q$ ; else  $d_{k+1} \leftarrow -g_{k+1}$ 
19:  end if
20: end for

```

Backup — perturbed accelerated gradient descent (Jin et al. '18)

- 1: **Parameters:** $\eta = \frac{1}{4\ell}$, $\theta = \frac{1}{4\sqrt{\kappa}}$ with $\kappa = \ell/\sqrt{\rho\varepsilon}$, $\gamma = \theta^2/\eta$, $s = \gamma/(4\rho)$, radius r , window $\mathcal{T}$
 - 2: $x_0 \leftarrow x_{\text{init}}$, $v_0 \leftarrow 0$
 - 3: **for** $t = 0, 1, \dots$ **do**
 - 4: **if** $\|\nabla f(x_t)\| \leq \varepsilon$ and no perturbation in the last $\mathcal{T}$ steps **then**
 - 5: $x_t \leftarrow x_t + \xi$, $\xi \sim \text{Unif}(B_0(r))$
 - 6: **end if**
 - 7: $y_t \leftarrow x_t + (1 - \theta) v_t$ ▷ extrapolate
 - 8: $x_{t+1} \leftarrow y_t - \eta \nabla f(y_t)$
 - 9: $v_{t+1} \leftarrow x_{t+1} - x_t$
 - 10: **if** $f(x_t) \leq f(y_t) + \langle \nabla f(y_t), x_t - y_t \rangle - \frac{\gamma}{2} \|x_t - y_t\|^2$ **then** ▷ negative curvature detected
 - 11: $(x_{t+1}, v_{t+1}) \leftarrow \text{NCE}(x_t, v_t, s)$
 - 12: **end if**
 - 13: **end for**
 - 14: **NCE**(x_t, v_t, s): **if** $\|v_t\| \geq s$, **return** $(x_t, 0)$; **else** $\delta = s \|v_t\| / \|v_t\|$ and **return** $(\arg \min_{\pm} f(x_t \pm \delta), 0)$
- The test on line 10 fires exactly when the Hamiltonian **fails to decrease**. NCE then kills the momentum and forces the decrease by hand — which is what makes a momentum coefficient $1 - \theta$ **close to 1** safe.
 - $\theta = \frac{1}{4\sqrt{\kappa}}$ with $\kappa = \ell/\sqrt{\rho\varepsilon}$, so the gain over GD is $\sqrt{\kappa} \sim \varepsilon^{-1/4}$, and $\varepsilon^{-2}/\varepsilon^{-1/4} = \varepsilon^{-7/4}$.

Backup — why the dynamical-systems argument fails for NCG

The recipe (Lee et al.)

Write the method as $x_{k+1} = G(x_k)$. If G is a **local diffeomorphism**, the Center-Stable Manifold Theorem confines the initialisations converging to a strict saddle to a set of measure zero. For GD: $G(x) = x - \alpha \nabla f(x)$, $DG = I - \alpha \nabla^2 f(x)$, invertible for $\alpha < 1/L_1$. **Done.**

For NCG the state is the **pair** $z_k = (x_k, d_k) \in \mathbb{R}^{2n}$:

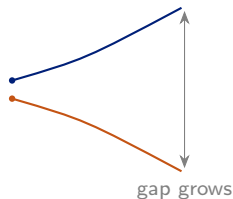
$$G(x, d) = (x + \alpha d, -\nabla f(x + \alpha d) + \beta(x, d) d), \quad \text{fixed points } (x^*, 0), \quad \nabla f(x^*) = 0.$$

Three separate obstructions

- (i) β is a **ratio of gradients**. At a fixed point $d = 0$, so $x_{k+1} = x_k$ and $\beta^{\text{FR}} = \|g_{k+1}\|^2 / \|g_k\|^2$ is $0/0$: G is not even defined at the point where the theorem is applied.
- (ii) $\beta^{\text{PRP+}} = \max\{\cdot, 0\}$ is **not** C^1 across the truncation.
- (iii) Restarts and the line search make the map **change at every iteration**, with $\alpha_k = \theta^{j_k}$ discontinuous in the state. The classical theorem needs one fixed smooth map.

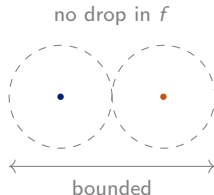
Non-autonomous versions exist (Muşat & Boumal), but (i) and (ii) are about β , not about the schedule.

Backup — how the proof goes, in three pictures

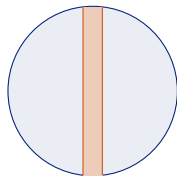


1. Two runs started a little apart along the escape direction **pull apart geometrically**, at a rate $\mu_+ > 1$ set by the curvature.

The scheme is that of Jin et al. '17–'18.



2. A run that fails to lower f **stays in a small ball**. Two failing runs cannot be far apart: it's a contradiction so they cannot both fail.



3. The failing perturbations therefore form a **thin slab**, and a uniform draw misses it with high probability.

Backup — why the perturbation argument fails for NCG

Two runs from ξ and $\xi' = \xi + \ell_0 v_{\min}$. Everything rests on the difference $w_t = (\Delta x_t, \alpha_w \Delta d_{t-1})$ obeying a **linear** recursion with a **fixed** matrix, and a remainder proportional to the gap:

$$w_{t+1} = Mw_t + u_t, \quad \|u_t\| \leq \chi \|w_t\|.$$

What a line search does to this

Differencing $x_{t+1} = x_t + \alpha_t d_t$ and $x'_{t+1} = x'_t + \alpha'_t d'_t$:

$$\Delta x_{t+1} = \Delta x_t + \alpha_t \Delta d_t + \underbrace{(\alpha_t - \alpha'_t) d'_t}_{\text{order } \|d'_t\|, \text{ not order } \|w_t\|}.$$

Armijo returns $\alpha_t = \theta^i$: one extra backtrack in one run makes $\alpha_t - \alpha'_t$ jump by a factor θ **however close the runs are**, and $\|u_t\| \leq \chi \|w_t\|$ fails at the first induction step.

- A per-run β gives the same obstruction through $(\beta_t - \beta'_t) d'_{t-1}$. **Restart is the extreme case**: if the two runs disagree, $\beta_t - \beta'_t = \beta_t$.
- Even with parameters *shared* but time-varying, M becomes a sequence M_t , and iterating gives the product $M_{T-1} \cdots M_0$ instead of M^T . Powers of one matrix are controlled by its eigenstructure; arbitrary products are not.

Backup — the escape bound in full, and where δ enters

- ❶ δ enters once, in the window length needed for the volume argument:

$$T \geq \frac{1}{\log \mu_+} \log \left[\frac{1}{c_{\text{esc}} - c c_h} \left(1 + \frac{2S}{\delta R} \sqrt{\frac{n+1}{2\pi}} \right) \right], \quad \log \mu_+ \geq \frac{\varepsilon_H}{2L_1}.$$

- ❷ Resulting scales, with $\phi := 1 + \log(nL_1L_2\Delta_f/(\varepsilon_H\delta))$:

$$T = O\left(\frac{L_1\phi}{\varepsilon_H}\right), \quad S = \Omega\left(\frac{\varepsilon_H}{L_2\phi}\right), \quad F = \Omega\left(\frac{\varepsilon_H^3}{L_2^2\phi^3}\right).$$

- ❸ Budget and union bound. Each surviving window burns $F_{\text{cert}} = (1 - \omega)F$ of Δ_f , so

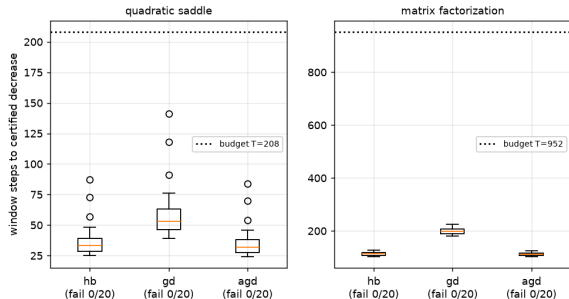
$$\bar{K}_{\max} = \frac{\Delta_f}{(1 - \omega)F} + 1 = O\left(\frac{L_2^2\Delta_f\phi^3}{\varepsilon_H^3}\right), \quad \delta = \frac{\delta_{\text{tot}}}{\bar{K}_{\max}}.$$

- ❹ Total window iterations. $\phi_0 := 1 + \log(nL_1L_2\Delta_f/(\varepsilon_H\delta_{\text{tot}}))$:

$$|\mathcal{W}| \leq T \bar{K}_{\max} = O\left(\frac{L_1L_2^2\Delta_f\phi_0^4}{\varepsilon_H^4}\right).$$

δ_{tot} appears only inside ϕ_0 , so the cost of the failure probability is $\log^4(1/\delta_{\text{tot}})$, and the dimension enters the same way, through $\log^4 n$.

Backup — window length and window subroutine



Cutting the window by a factor two to four is safe. The safe factor differs between the two problems, and finding it requires the Hessian.

Replacing heavy ball inside the window by gradient descent is slower; by accelerated gradient, no faster. At a momentum coefficient below $\frac{1}{2}$ that is what the analysis predicts.

Backup — what is known in the stochastic setting

setting	algorithm	iterations	guarantee
non-stochastic	GD	$\mathcal{O}(\epsilon^{-2})$	FOSP
	PGD	$\tilde{\mathcal{O}}(\epsilon^{-2})$	SOSP
stochastic	SGD	$\mathcal{O}(\epsilon^{-4})$	FOSP
	PSGD	$\tilde{\mathcal{O}}(\epsilon^{-4})$	SOSP
	PSGD, without per-sample smoothness	$\tilde{\mathcal{O}}(d \epsilon^{-4})$	SOSP

Jin, Netrapalli, Ge, Kakade & Jordan, *On nonconvex optimization for machine learning: gradients, stochasticity, and saddle points*, JACM 2021.

Second-order stationarity costs only a **polylogarithmic factor** over first-order, in both settings.
The dimension enters only if each stochastic gradient is not assumed Lipschitz.